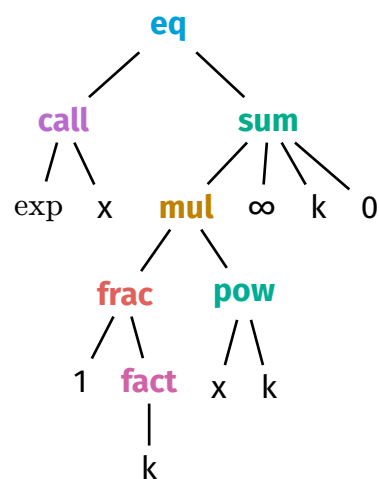


Parse equations with Typst

$$\exp(x) = \sum_{k=0}^{\infty} \frac{1}{k!} x^k$$



```
eq(call(exp, x), sum(k, 0, ∞, mul(frac(1, fact(k)), pow(x, k))))
```



1. At a glance	3
2. Usage examples	4
3. Guide	8
4. Function reference	19

Contents

1. At a glance	3
2. Usage examples	4
2.1. Drawing expression trees	4
2.2. Evaluating Boolean formulae	5
2.3. Plotting equations	6
2.4. Evaluating dimensionful quantities with Pariman	7
3. Guide	8
3.1. Declaring grammars	8
3.1.1. Common grammars	8
3.2. Declaring operators	9
3.2.1. Prefix, infix, postfix and match operators	9
3.2.2. Pattern matching and slots	9
3.2.2.1. Matching sequences greedily or lazily	10
3.2.2.2. Matching any pattern in a union	10
3.2.2.3. Matching conditionally (slot guards)	11
3.2.2.4. Matching whitespace tightly or loosely	11
3.2.3. Operator precedence	12
3.2.4. Associativity of infix operators	13
3.2.5. Operator predicates (operator guards)	13
3.2.6. Operator rewrite rules	14
3.3. How trees are represented	15
3.3.1. Tree traversal	15
3.4. Gotchas	16
3.4.1. Parsing juxtaposition as an operator	16
3.4.2. Parsing exponents and subscripts	17
4. Function reference	19
Module <code>parsely</code>	19
<code>parse(it, grammar, min-prec)</code>	19
Module <code>parsely.render</code>	19
<code>render.tree(tree, grow, spread, ..)</code>	19
<code>render.waterfall(tree, inset, grow, ..)</code>	20
Module <code>parsely.util</code>	21
<code>util.walk(tree, pre, post, leaf)</code>	21
<code>util.element-fields-to-arguments(fn, fields)</code>	21
<code>util.stringify(it)</code>	22
<code>util.content-to-tree(it, exclude)</code>	22
<code>util.tree-to-content(tree)</code>	22
<code>util.random-color(seed, lightness, chroma, hue)</code>	22

1. At a glance

1. Import the package with:

```
#import "@preview/parsely:0.1.1"
```

2. **Declare a grammar** by listing the mathematical **operators** you want to parse:

```
#import parsely: slot
#let grammar = (
  eq: (infix: $=$, prec: 0),
  add: (infix: $+$, prec: 1, assoc: true),
  mul: (infix: $$, prec: 2, assoc: true),
  grp: (match: $(slot("body*"))$),
  pow: (match: $slot("base")^slot("exp")$),
)
```

Operators can be given **precedence**, **associativity** and **pattern matching slots**.

Alternatively, use the built-in grammar, `parsely.common.arithmetic`.

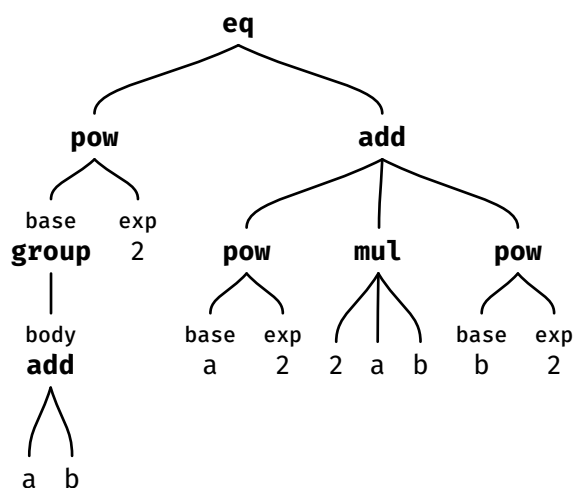
3. Call `parsely.parse()` on content along with the grammar to use.

```
#let eqn = $(a + b)^2 = a^2 + 2a b + b^2$
#let (tree, rest) = parsely.parse(eqn, grammar)
```

This returns a syntax tree along with any trailing tokens that failed to parse (if any).

4. Use `parsely.walk()` or `parsely.render()` to visit nodes and turn them into content.

```
#parsely.render.tree(tree)
```



```
#parsely.walk(tree, post: n => parsely.render.node(n, grammar))
```

$(a + b)^2 = a^2 + 2ab + b^2$

2. Usage examples ↑

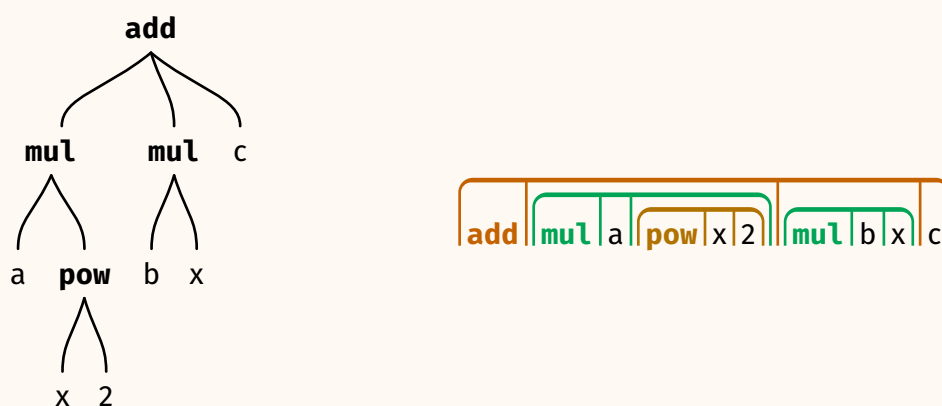
This section contains interesting self-contained applications of Parsely. Each section is a standalone example file which may be found at <https://github.com/Jollywatt/typst-parsely/blob/v0.1.1/docs/examples>.

Source code: [examples/trees.typ](#)

2.1. Drawing expression trees

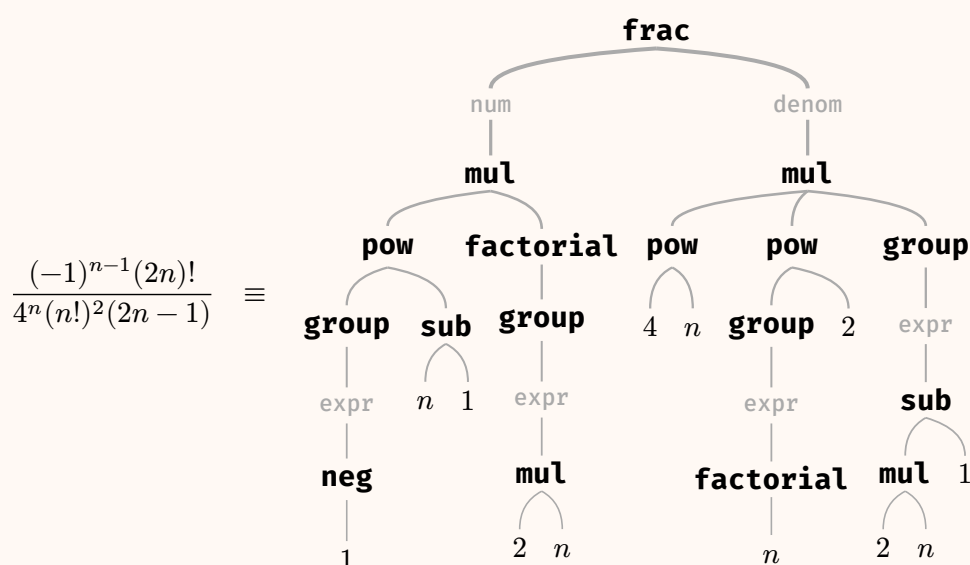
You can use the built-in functions `tree()` and `waterfall()` in the `parsely.render` module to quickly visualise syntax trees.

For example, here is the syntax tree for the equation $ax^2 + bx + c$ parsed with the grammar `parsely.common.arithmetic` and rendered as a tree and waterfall:



The `CeTZ` package can also draw trees represented as nested arrays. You can transform Parsely's syntax trees from their dictionary form into array form (`node, .. children`) with a simple post-walk.

For example, the n th coefficient in the series expansion of $\sqrt{1+x}$ is:



Source code: [examples/venn.typ](#)

2.2. Evaluating Boolean formulae [↑]

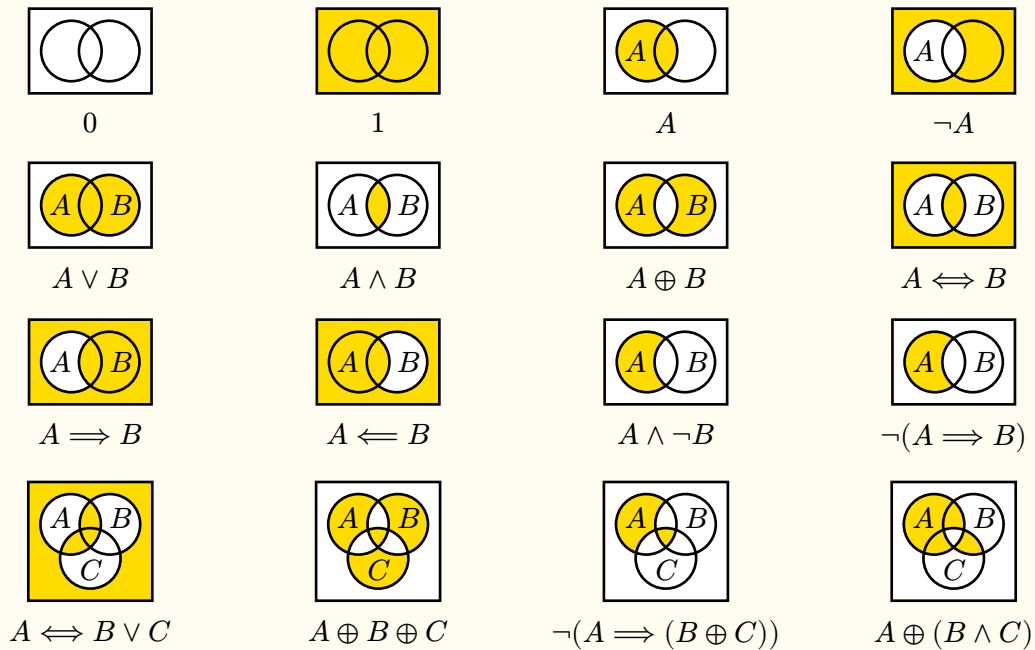
This example demonstrates how equations may be evaluated by parsing them and then converting the syntax tree into anonymous functions, which we can call with dictionary defining the equation's unknowns.

We define a simple grammar to parse Boolean operators which also defines how the operators are evaluated. For example, the and operator dictionary (`infix: $and$, prec: 2, eval: (a, b) ⇒ a and b`) has an `eval` field. We evaluate and then plot the results of various formulae using `CeTZ` and its `cetz-venn` library.

After parsing, we perform a tree walk to transform the syntax tree into a callable function. First, leaf nodes are transformed into anonymous functions accepting a dictionary of values. For example, the leaf node “A” is mapped to `scope ⇒ scope.at("A")`.

Then, operators are transformed into functions which first evaluate their arguments and then combine them with some operation. For example, the partially-evaluated node “left $\wedge$ right” becomes `scope ⇒ left(scope) and right(scope)`.

The final closure accepts a scope such as (`"A": true, "B": false`). By testing on various inputs, you can then use the closure to plot Venn diagrams or a truth table.



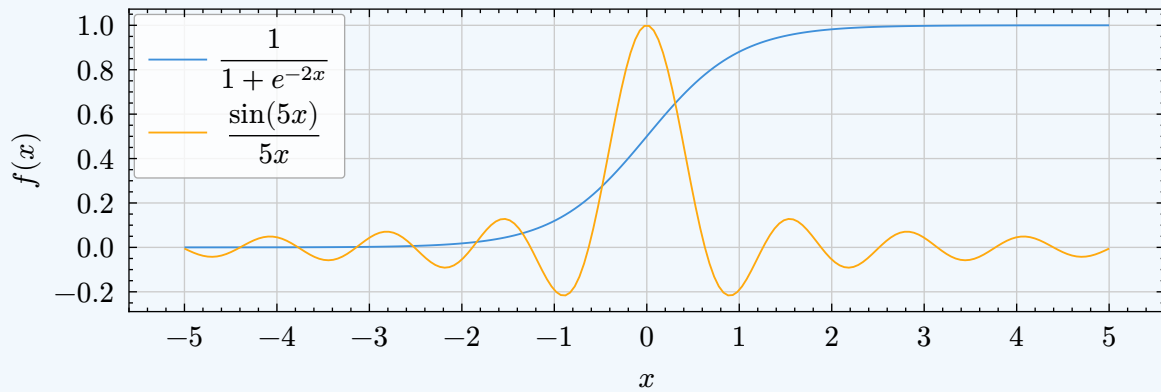
A	B	¬A	A ∨ B	A ⊕ B	A ∧ B	A ⇔ B	A ⇒ B	A ⇐ B	A ∧ ¬B	1
0	0	1	0	0	0	1	1	1	0	1
1	0	0	1	1	0	0	0	1	1	1
0	1	1	1	1	0	0	1	0	0	1
1	1	0	1	0	1	1	1	1	0	1

Source code: [examples/calc.typ](#)

2.3. Plotting equations ↑

This example, similar to **Evaluating Boolean formulae**, shows how equations may be converted into anonymous functions which can be evaluated by passing a dictionary of values of the equation's unknowns. Plots are produced with the beautiful [Lilag](#) package.

For example, $\frac{1}{1+e^{-2x}}$ and $\frac{\sin(5x)}{5x}$ are typed once yet displayed and plotted automatically.



First, an equation is parsed using the built-in `parsely.common.arithmetic` grammar, resulting in a syntax tree. To convert the tree into an anonymous function, leaf nodes like 4.5, π or `cos` are first mapped to anonymous functions which return a constant like `s  $\Rightarrow$  eval("4.5")`, `s  $\Rightarrow$  calc.pi` or `s  $\Rightarrow$  calc.cos` (yes, in the last case a function is returned as the value).

Then, nodes are also converted into anonymous functions which accept a scope dictionary and return a value. For example, $x + y$ becomes

```
s  $\Rightarrow$  args.first()(s) + args.last()(s)
```

where `args.first()` is `s  $\Rightarrow$  s.at("x")`.

Function application such as $\cos(x)$ becomes

```
s  $\Rightarrow$  (slots.op)(s)((slots.args)(s))
```

where the value of `slots.op(s)` is the actual function `calc.cos`.

This might be hard to follow, but don't worry; look at the code.

Source code: <examples/pariman.typ>

2.4. Evaluating dimensionful quantities with Pariman

You can parse equations with Parsely and evaluate them with Pariman by writing a tree post-walk which converts each leaf into a `pariman.quantity` and evaluates nodes using `pariman.calculation` functions.

For example, here is an equation which we first parse then substitute dimensionful quantities into:

$$\begin{aligned}\frac{1}{2}mv^2 &= \frac{1}{2} \times 75 \text{ kg} \times (1.5 \text{ m s}^{-1})^2 \\ &\approx 84 \text{ kg m}^2 \text{ s}^{-2}\end{aligned}$$

The following examples use Pariman's *in-text quantity declarations*, letting you define quantities and derived values with normal equation syntax.

2.4.1. Chemical concentration

I put $M := 30.0 \text{ g}$ of sugar into $V := 105 \text{ mL}$ of water in a cup. After being stirred thoroughly, the sugar solution will have a concentration of:

$$\begin{aligned}c &:= \frac{M}{V} = \frac{30.0 \text{ g}}{105 \text{ mL}} \\ &\approx 0.286 \text{ g mL}^{-1}\end{aligned}$$

2.4.2. Terminal velocity

A bowl of petunias of mass $m := 1.1 \text{ kg}$ pops into existence at $h := 12\,000 \text{ m}$ altitude. Taking gravity to be $g := 9.81 \text{ m s}^{-2}$ (assuming this is still Earth) the petunias have gravitational potential energy:

$$\begin{aligned}U &:= mgh = 1.1 \text{ kg} \times 9.81 \text{ m s}^{-2} \times 12\,000 \text{ m} \\ &\approx 1.3 \times 10^5 \text{ kg m}^2 \text{ s}^{-2}\end{aligned}$$

Ignoring air resistance, the petunias smash into the ground at velocity:

$$\begin{aligned}V &:= \sqrt{2gh} = (2 \times 9.81 \text{ m s}^{-2} \times 12\,000 \text{ m})^{\frac{1}{2}} \\ &\approx 485 \text{ m s}^{-1}\end{aligned}$$

3. Guide

3.1. Declaring grammars

Grammars define how content is transformed into an abstract syntax tree.

A grammar is given as a dictionary where each value is an *operator* and each key is the operator's name, which becomes the name of the corresponding node in the syntax tree.

For example, the simple grammar in Listing 1 defines:

- the token “+” as an associative binary operator of lower precedence than “×” so that $a + b \times c$ is parsed as $a + (b \times c)$
- the token “−” as left associative so $a - b - c$ is parsed as $(a - b) - c$
- the **slot pattern** “base^{exp}” matching expressions like 2^5 , $(a + b)^2$ or $e^{-i(kx + \omega t)}$

```
#let grammar = (  
  add: (infix: $+$, prec: 1, assoc: true),  
  sub: (infix: $-$, prec: 1, assoc: left),  
  neg: (prefix: $-$, prec: 1),  
  mul: (infix: $times$, prec: 2, assoc: true),  
  pow: (match: $slot("base")^slot("exp")$),  
)
```

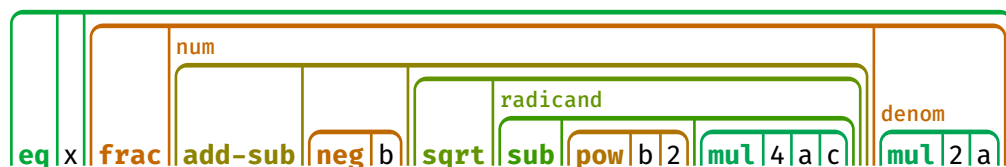
Listing 1: A simple example of a grammar

In a grammar, **the order that operators are listed matters**: the first matching operator in the grammar is used to parse a node. (This is separate to **operator precedence**.) This means operators should generally be listed with the more specific patterns earlier and “catch all” patterns later. (An important example of this is when **Parsing juxtaposition as an operator**.)

3.1.1. Common grammars

A built-in grammar `parsely.common.arithmetic` is defined in `src/common.typ`. This grammar is good for basic algebraic expressions and should serve as a starting point for more complex grammars for specific domains.

```
#let eq = $x = (-b plus.minus sqrt(b^2 - 4 a c))/(2a)$  
#let (tree, rest) = parsely.parse(eq, parsely.common.arithmetic)  
#parsely.render.waterfall(tree)
```



If your application requires a common grammar which you believe should be included in `parsely`, please contribute it by way of [GitHub Issue](#) or [Pull Request](#)!

3.2. Declaring operators

Operators correspond to the different entities that you want to parse. The values in a grammar dictionary define the operators, and each operator is defined by a dictionary of properties. The first key–value pair specify the operator’s **kind** and **pattern**. Additionally, an operator can have any of the properties listed in Table 1.

Operator key	Description
(kind)	For example, the operator (<code>infix: \$+\$, ..</code>) has the kind “infix” and the pattern <code>\$+\$</code> . This must be first entry in the operator dictionary.
<code>prec</code>	The Operator precedence level. Has no effect for match operators.
<code>assoc</code>	Associativity for infix operators, ignored for other kinds.
<code>guard</code>	A predicate function to allow matching nodes conditionally depending on the values of slots. Must be a boolean function accepting a dictionary.
<code>rewrite</code>	A rewrite rule or function which takes the node which would be produced and transforms it before parsing continues.
(others)	Any other fields are allowed and ignored.

Table 1: Meanings of entries in an operator dictionary.

3.2.1. Prefix, infix, postfix and match operators

An operator is specified as a dictionary whose first key is the **operator type** and first value is the **operator pattern**. For example, (`infix: $+$, ..`) is an operator of type *infix* with pattern `$+$`. There are four kinds of operators: *prefix*, *infix*, *postfix* and *match*.

OPERATOR TYPE	Prefix	Infix	Postfix	Match
POSITIONAL ARGUMENTS	one after	to either side	one before	none
SLOT ARGUMENTS	yes	yes	yes	yes
PRECEDENCE, <code>prec</code>	required	required	required	not applicable
ASSOCIATIVITY, <code>assoc</code>	not applicable	left/right/both	not applicable	not applicable

Table 2: Kinds of operators available when declaring grammars and their features.

Prefix, infix and postfix operators consume tokens around them as *positional arguments*, subject to **precedence**. Match operators do not consume tokens to the left or right, but simply match a pattern. All operators support **Pattern matching and slots**, possibly consuming tokens as *slot arguments*.

3.2.2. Pattern matching and slots

Operator patterns are used to match sequences of tokens while parsing content. Patterns can be:

- **Single tokens**, like `$+$` or `$in$`.
- **Sequences of tokens**, like `$::$`, `$="def"$` or `$dif/(dif x)$`.
- **Slot patterns**, like `$sum_slot("var")$` or `[$slot("left*"), slot("right*")][$`.

- **Element functions**, like `math.frac`, as a shorthand for the slot pattern matching that element and capturing its fields, i.e. `$frac(slot("num"), slot("denom"))$`.

Pattern matching is done by the function `parsely.match(pattern, expr)`, which returns a dictionary if the match is successful and `false` otherwise.

```
#import parsely: slot
- #parsely.match($1 + slot("rhs"), $1 + x^2$)
- #parsely.match($A B C$, $A B Omega$)
- #parsely.match(math.frac, $1/2$) // or $slot("num")/slot("denom")$

• (rhs: attach(base: [x], t: [2]))
• false
• (num: [1], denom: [2])
```

Slots are wildcard tokens that match content in several ways:

- `slot(name)` matches any single token.
- `slot(name, many: true, greedy: bool)` **matches any sequence**.
- `slot(name, any: array)` **matches any one of a set of patterns**.
- `slot(name, guard: function)` **matches a token conditionally**.
- The special patterns `parsely.tight` and `parsely.loose` allow **matching whitespace**.

3.2.2.1. Matching sequences greedily or lazily ↑

A slot such as `slot("rhs")` will match a single token, but **multiple tokens** can be matched with `slot("rhs", many: true)` or `slot("rhs*")` for short.

```
#parsely.match($1; 2; slot("etc"), $1; 2; 3; 4$) // single token slot
#parsely.match($1; 2; slot("etc*"), $1; 2; 3; 4$) // multi token slot

(etc: [3])(etc: sequence([3], [;], [ ], [4]))
```

By default, multi-token slots are **greedy**, preferring to match more tokens when there is choice. Conversely, **lazy** slots such as `slot("name*", greedy: false)` or `slot("name*?")` for short match as few tokens as possible.

```
- #parsely.match($slot("greedy*"), slot("rest*"), $alpha, beta, gamma$)
- #parsely.match($slot("lazy*?"), slot("rest*"), $alpha, beta, gamma$)

• (greedy: sequence([α], [;], [ ], [β]), rest: [γ])
• (lazy: [α], rest: sequence([β], [;], [ ], [γ]))
```

3.2.2.2. Matching any pattern in a union ↑

Slots with an “any” argument containing an array of sub-patterns only match one of those patterns (in the order they are given).

This is sometimes useful for grouping many similar tokens together into one operator.

Operators in grammar

```
comp: (infix: slot("op", any: ($=$, $≠$, $<$, $>$, $≤$, $≥$)))
```

Parsing examples

$\xi = 2 > \varepsilon \neq 0 \mapsto$ 

Example 2: All comparison tokens are parsed as the "comp" operator tagged by an "op" slot.

3.2.2.3. Matching conditionally (slot guards) [↑](#)

Slots can be made conditional by supplying a boolean predicate in the "guard" argument. This is a function accepting the slot's matched content and returning a boolean. You can also use a regular expression instead of a function. A regular expression guard is shorthand for the predicate function `it ⇒ parsely.util.stringify(it).match(re) ≠ none`.

Operators in grammar

```
sep: (infix: $,$, assoc: true),  
number: (match: slot("it", guard: regex("^[\d\.\.]+$"))),  
text: (match: slot("it", guard: it ⇒ it.func() = text)),
```

Parsing examples

$\pi, \text{hi}, \bigcirc^2, 1.414 \mapsto$ 

3.2.2.4. Matching whitespace tightly or loosely [↑](#)

The presence of whitespace in equations is not always visible (for example, `$f(x)$` and `$f (x)$` are rendered identically) and whitespace is usually ignored when pattern matching. However, the presence or lack of whitespace between tokens can be explicitly matched with the special `parsely.tight` and `parsely.loose` patterns. For example, you can write a pattern that matches `$k!$` but not `$k !$` by using `tight`:

```
#import parsely: tight, loose  
#parsely.match($slot("a") !$, $A!$) // matches (insensitive)  
#parsely.match($slot("a") !$, $A !$) // matches (insensitive)  
#parsely.match($slot("a") tight !$, $A!$) // matches  
#parsely.match($slot("a") loose !$, $A !$) // matches  
#parsely.match($slot("a") tight !$, $A !$) // no match (too loose)  
#parsely.match($slot("a") loose !$, $A!$) // no match (too tight)
```

(a: [A])(a: [A])(a: [A])(a: [A]) false false

This can be useful to disambiguate function application `$f(x, y)$` from implicit multiplication, `$x (1 - y)$`, for example.

Operators in grammar

```
fact: (postfix: $tight !$, prec: 3),
assert: (postfix: $loose !$, prec: 0),
mul: (infix: $$, prec: 2, assoc: true),
grp: (match: $(slot("body*"))$),
call: (match: $slot("fn") tight (slot("body*"))$),
pow: (match: $slot("base")^slot("exp")$),
```

Parsing examples

$\$lambda\ f(x^2)\$ \mapsto$ 

$\$lambda\ f\ (x^2)\$ \mapsto$ 

$\$n\ k!\$ \mapsto$ 

$\$P\ Q\ !\$ \mapsto$ 

Example 4: Tightness and looseness are used to distinguish call operators from mul's. Similarly, fact and assert are distinguished by whitespace patterns.

3.2.3. Operator precedence [↑]

Operators which consume positional arguments (prefix, infix or postfix operators) have an optional precedence specified by a `prec` key which controls how tightly they bind to operands (neighbouring non-whitespace tokens). The default precedence is zero.

Operators in grammar

```
add: (infix: $+ $, prec: 1, assoc: true),
neg: (prefix: $- $, prec: 2),
dot: (infix: $dot $, prec: 1.5),
fact: (postfix: $! $, prec: 3),
query: (postfix: $? $, prec: 0),
mul: (infix: $$, prec: 2, assoc: true),
```

Parsing examples

$-x + u \cdot \lambda v + pqr \mapsto [[[-x] + [u \cdot [\lambda v]]] + [pqr]]$

$4\pi r^2 + z! \mapsto [[4\pi r^2] + [z!]]$

$-Xk!Z? \mapsto [[-X[k!]Z]]?$

Example 5: Precedence is higher for “stickier” operators, and can be an integer or float.

All operators support slot patterns, in particular prefix operators. This allows summation notation “ $\sum_{\text{var}} \text{body}$ ” to be parsed as a prefix operator with slots, for example.

Operators in grammar

```
add: (infix: $+$, prec: 1, assoc: true),
dot: (infix: $dot$, prec: 2),
sum: (prefix: $sum_slot("var")$, prec: 2),
mul: (infix: $$, prec: 2, assoc: true),
grp: (match: $(slot("body*"))$),
```

Parsing examples

$$\begin{aligned}\sum_i \alpha x_i + y + z &\mapsto \left[\left[\sum_i [\alpha x_i] \right] + y + z \right] \\ \sum_i \alpha (x_i + y_i) + z &\mapsto \left[\left[\sum_i [\alpha [(x_i + y_i)]] \right] + z \right] \\ \sum_i \alpha (x_i \cdot y_i + z_i) &\mapsto \left[\sum_i [\alpha [(x_i \cdot y_i) + z_i]] \right]\end{aligned}$$

Example 6: Summation notation as a prefix operator with higher precedence than addition.

3.2.4. Associativity of infix operators [↑]

Infix operators additionally have an associativity specified by an `assoc` key which applies when the same operator appears in a sequence. Possible values are `left` (default), `right` and `true`, for left/right associativity and true associativity (meaning the operator merges with itself and collects multiple arguments) respectively.

Operators in grammar

```
left: (infix: $←$, assoc: left), // always has exactly two args
right: (infix: $⇒$, assoc: right), // always has exactly two args
both: (infix: $⇔$, assoc: true), // can have more than two args
```

Parsing examples

$$\begin{aligned}a \leftarrow b \leftarrow c &\mapsto [[a \leftarrow b] \leftarrow c] \\ a \Rightarrow b \Rightarrow c &\mapsto [a \Rightarrow [b \Rightarrow c]] \\ a \Leftrightarrow b \Leftrightarrow c &\mapsto [a \Leftrightarrow b \Leftrightarrow c]\end{aligned}$$

3.2.5. Operator predicates (operator guards) [↑]

Similar to **slot guards**, operators can be given guards which match depending on all the matched slots. Guards can be specified by adding a `guard` entry to the operator dictionary. The predicate function for a slot guard receives a dictionary of all slots as its only argument.

Example 8 shows how you can use an operator guard to match integrals conditionally if neither of the limits contain the string " ∞ ".

Operators in grammar

```
definite-integral: (
  prefix: $integral_slot("lo")^slot("hi")$,
  guard: slots => {
    not slots.values().any(it => "∞" in parsely.stringify(it))
  }
),
integral: (prefix: $integral_slot("lo")^slot("hi")$),
neg: (prefix: $-$, prec: 2),
mul: (infix: $$, prec: 2, assoc: true), // must be before dif
dif: (prefix: $dif$, prec: 3),
```

Parsing examples



Example 8: Using operator guards to only match integrals that don't have "∞" in the limits.

3.2.6. Operator rewrite rules ↑

Operator rewriting is an advanced feature which is sometimes necessary when the desired syntax tree has a sufficiently different topology from the content tree emitted by Typst. For example, correctly **Parsing exponents and subscripts** requires a rewrite rule.

An operator rewrite rule is a function which transforms a node immediately after the operator is matched but before its children are parsed. Rewrite rules can be specified by adding a `rewrite` entry to the operator dictionary. The rewrite rule function receives a node dictionary whose `args` and `slots` are yet to be parsed. The rule may return a node, in which case parsing continues normally on any child nodes.

Be aware that parsing with rewrite rules is prone to infinite recursion if the rule returns unparsed content which itself matches the operator.

Example 9 shows how a rewrite rule can be used to convert slots arguments into named arguments.

Operators in grammar

```
add: (infix: $+$),
slot-pair: (
  match: ${slot("left*"), slot("right*")}$,
),
arg-pair: (
  match: ${slot("left*"), slot("right*")}$,
  rewrite: ((head, args, slots)) => {
    (head: head, args: (slots.left, slots.right), slots: (:))
  }
)
```

Parsing examples

$\{A, B + C\} \mapsto$ 

$[A + B, C] \mapsto$ 

Example 9: Using an operator rewrite rule to convert slot a

3.3. How trees are represented [↑]

Everywhere in this manual, a “tree” or “node” refers to a dictionary of the form

```
(head: str, args: array, slots: dictionary)
```

where “head” is name of the node, and the node’s children are the elements of “args” or the values of “slots”. Both args and slots must be present, even if they are empty.

Nodes in a tree are often represented in the simpler array form (head, .. children). However, having both kinds of children makes it easier to represent

- operators with both positional arguments and named slots; and
- element functions associated with positional and named arguments.

3.3.1. Tree traversal

You can do many things with a syntax tree with `util.walk(tree, pre, post, leaf)`. For example, Listing 10 implements a post-walk which transforms leaf nodes (the symbols *a*, *b* and 2) into “{a}”, “{b}”, “{2}” and then converts operator into strings with custom rules.

```
#let tree = parsely.parse($sqrt(a + b^2)$,
parsely.common.arithmetic).tree
#parsely.walk(tree, post: node => {
  let (head, args, slots) = node
  if head == "add" { args.join(" + ") }
  else if head == "pow" { args.join("^") }
  else if head == "sqrt" { "√(" + slots.radicand + ")" }
  else { repr(node) } // a fallback representation
}, leaf: it => "{" + it + "}")
```

$\sqrt{\{a\} + \{b\}^{\{2\}}}$

Listing 10: Traversing the syntax tree to output a string.

Similar post-order tree walks can be used to rewrite nodes, reorder arguments, evaluate expressions numerically, or return content with certain styles or annotations added to specific nodes.

See the source code of the [Usage examples](#) for many different examples of syntax tree manipulations with tree walks.

3.4. Gotchas [↑]

Here are some tips for achieving common things which are a little tricky to get exactly right.

3.4.1. Parsing juxtaposition as an operator

It is common to want to parse sequences of consecutive non-operator tokens. The default behaviour is to stop parsing when a token is encountered that is not the argument or slot of an operator. To parse multiple tokens as one, you can use strings (if the formatting is acceptable) or wrap the tokens in an invisible box.

Operators in grammar

```
op: (infix: $plus.o$),  
grp: (match: box),
```

Parsing examples

`$1 plus.o a b c$` $\mapsto$ `op` | 1 | `a` | `bc`

`$1 plus.o "abc"$` $\mapsto$ `op` | 1 | `abc`

`$1 plus.o #box($a b c$)$` $\mapsto$ `op` | 1 | `grp` | body | `abc`

Example 11: Different ways to parse multiple juxtaposed tokens. The trailing red symbols show the `rest` argument returned by `parseLy.parse()`, containing content that failed to parse.

Alternatively, juxtaposition can be parsed as an infix operator by using an empty pattern (`$$` or `none`). This is useful for parsing implicit multiplication, in which case the operator is also given a product-level precedence, as in Example 12.

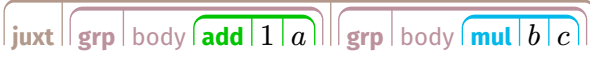
Because the empty pattern always matches anything, **juxtaposition operators should appear later than other infix operators** in the grammar dictionary (otherwise $a \times b$ is parsed as three tokens ($a, \times, b$) juxtaposed). Additionally, they should occur **before match operators**, otherwise trailing tokens will be encountered (and parsing halted) before the parser realises the tokens can be interpreted as the right-hand argument of the juxtaposition operator.

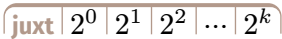
Operators in grammar

```
add: (infix: $+$, prec: 1),
mul: (infix: $times$, prec: 2), // "juxt" must be after this
juxt: (infix: none, assoc: true, prec: 2),
grp: (match: $(slot("body*"))$), // "juxt" must be before this
```

Parsing examples

$1 + ab \times c \mapsto$ 

$(1 + a)(b \times c) \mapsto$ 

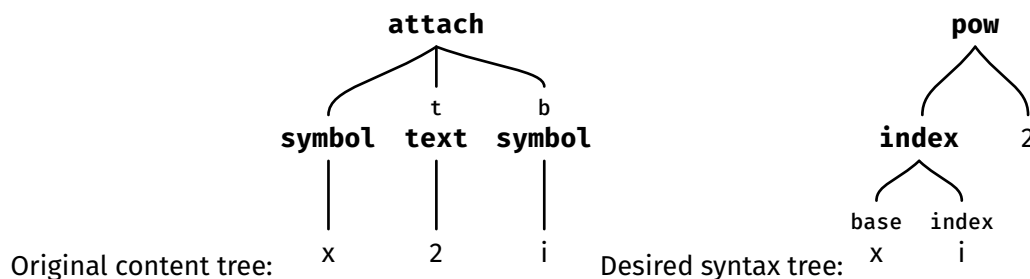
$2^0 2^1 2^2 \dots 2^k \mapsto$ 

Example 12: Parsing implicit multiplication.

3.4.2. Parsing exponents and subscripts ↑

Because Typst content is itself a tree-like structure, some content cannot be parsed in the same way that a string of tokens would be. For instance, the body of the equation x_i^2 is a single `math.attach` element containing both the superscript and subscript as fields:

```
#let tree = parsely.util.content-to-tree($x_i^2$.body)
Original content tree:
#parsely.render.tree(tree)
Desired syntax tree:
#parsely.render.tree((head: "pow", args: ((head: "index", args: ()),
slots: (base: [x], index: [i])), 2), slots: ()))
```



In fact, x_i^2 and x^2_i are indistinguishable. This makes parsing x_i^2 as $(x_i)^2$ as opposed to $(x^2)_i$ slightly tricky, for example.

To illustrate what why this is an issue, the grammar in Example 13 naively matches `math.attach` elements, but silently drops other fields such as subscripts, so x_i^2 becomes x^2 .

Operators in grammar

```
pow: (match: $slot("base")^slot("exp")$) // don't do this
```

Parsing examples

$x_i^2 \mapsto$ 

Example 13: A limitation of slot-based pattern matching. Notice the subscript gets dropped.

The trick is to use an operator **rewrite** rule, which lets us manipulate the node however we want as soon as the parser encounters content matching the operator's pattern. A rewrite rule is a function which accepts the node (as it would be parsed) and outputs content or another node which the parser will continue to descend into.

The **pow** operator in Example 14 matches **math.attach** elements, and uses a **rewrite** rule to ensure that non-superscript attachments are added to the base before parsing continues.

Operators in grammar

```
group: (match: $(slot("body*"))$),
add: (infix: $+$),
pow: (
  match: math.attach,
  guard: slots ⇒ "t" in slots, // required to avoid infinite recursion
  rewrite: it ⇒ {
    let (base, t, ..rest) = it.slots
    if rest.len() > 0 { base = math.attach(base, ..rest) }
    (head: "pow", args: (base, t), slots: (:))
  }),
```

Parsing examples

$\$x_i^2\$ \mapsto \text{pow } x_i \text{ } 2$
 $\$x^2_i\$ \mapsto \text{pow } x_i \text{ } 2$
 $\$x_0\$ \mapsto x_0$
 $\$(x^i + y_j)^{(p + q)}\$ \mapsto \text{pow } \text{group } \text{body } \text{add } \text{pow } x \text{ } i \text{ } y_j \text{ } \text{add } p \text{ } q$

Example 14: A correct way of parsing superscripts while preserving other attachments such as subscripts.

Rewrite rules can be dangerous in the same sense as a tree pre-walk: it can result in infinite recursion if the output of the rewrite rule is matched by the same operator.

To guard against this, we must use the **operator guard** `slots ⇒ "t" in slots` which ensures that **pow** only applies to **math.attach** elements which have a superscript (and possibly other fields).

4. Function reference

Module <code>parsely</code>	19
Module <code>parsely.render</code>	19
Module <code>parsely.util</code>	21

Module `parsely`

- [`parse\(\)`](#)

[`parse\(it, grammar, min-prec\)`](#)

Parse content into a tree according to a specified grammar, returning a dictionary (`tree`, `rest`) with the parsed syntax tree and any trailing content that failed to parse.

```
let (tree, rest) = parsely.parse($f(x) = x^2/2$, parsely.common.arithmetic)
```

Not all content in the syntax tree needs to be parsed; content that fails to be parsed is left as is, which may result in a “partially parsed” syntax tree with content in some nodes.

When is called on some content, the parser tries to match the content with operators defined in the grammar. If successful, the parser recursively descends into arguments and tries to parse those. If parsing slot arguments fails, the slot is simply left as content in the resulting syntax tree. If parsing a positional argument fails, what was parsed so far is returned in `tree` and remaining content is returned in `rest`.

Module `parsely.render`

- [`render.tree\(\)`](#)
- [`render.waterfall\(\)`](#)

[`render.tree\(tree, grow, spread, ..\)`](#)

Render a tree given in the format described in [How trees are represented](#).

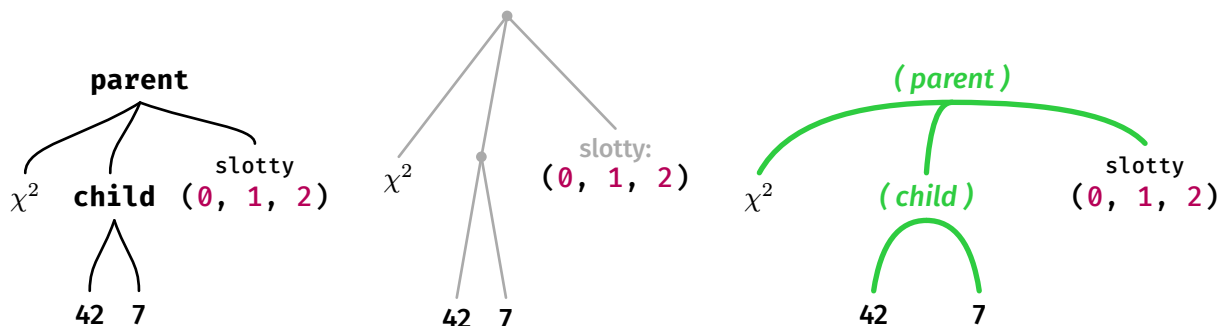
- **grow:** `2.5em` Height between generations
- **spread:** `1em` Distance between siblings
- **inset:** `0.5em` Gap between nodes and edges
- **head-style:** `it` $\Rightarrow$ `strong(raw(it))` Style for node names
- **slot-style:** `it` $\Rightarrow$ `text(0.8em, raw(it))` Style for slot labels

- **edge:** `curve(curve.cubic((50%, 50%), (100%, 60%), (100%, 100%)))` Curve to draw between nodes. This gets placed in the tree so that the start point is (0%, 0%) and end is (100%, 100%). For example:

```
line(start: (0%, 0%), end: (100%, 100%), stroke: blue) // straight
curve(curve.cubic((50%, 50%), (100%, 60%), (100%, 100%))) // brace
curve(curve.quad((100%, 0%), (100%, 100%)), stroke: 2pt) // arc
```

This is meant to be useful for debugging; for more control, consider using the `cetz.tree` submodule of `CeTZ`.

```
let tree = (
  head: "parent",
  args: ($chi^2$, (head: "child", args: (42, 7), slots: (:))),
  slots: (slotty: range(3)),
)
parsely.render.tree(tree)
h(1fr)
parsely.render.tree(tree, grow: 5em, inset: 4pt,
  head-style: it => pad(y: -6pt, circle(radius: 2pt, fill: gray)),
  slot-style: it => text(0.9em, gray)[#it:],
  edge: line(start: (0%, 0%), end: (100%, 100%), stroke: gray))
h(1fr)
parsely.render.tree(tree, spread: 3em,
  head-style: it => text(green, emph[( #it )]),
  edge: curve(curve.quad((80%, 0%), (100%, 100%)), stroke: 2pt + green))
```



`render.waterfall(tree, inset, grow, ..)` ↑

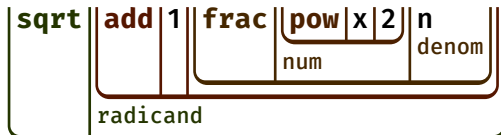
Visualise a tree as a sequence of nodes enclosed by over- or under-braces.

- **inset:** `0.25em` Padding inside nodes
- **grow:** `0.5em` Vertical padding; higher makes a taller diagram
- **radius:** `0.5em` Corner radius
- **head-color:** `util.random-color` Function taking the node's head and returning a color.
- **head-style:** `it => strong(raw(it))` Style for node names
- **slot-style:** `it => text(0.8em, raw(it))` Style for slot labels
- **side:** `top` Show braces on this side of nodes

```
let tree = parsely.util.content-to-tree(circle(stroke: 5pt)[Hello *World*])
parsely.render.waterfall(tree)
```



```
let tree = parsely.parse($sqrt(1 + x^2/n)$, parsely.common.arithmetic).tree
let c = parsely.util.random-color.with(lightness: (30%, 30%), chroma: 30%)
parsely.render.waterfall(tree, side: bottom, head-color: c)
```



Module `parsely.util` [↑](#)

- `util.walk()`
- `util.element-fields-to-arguments()`
- `util.stringify()`
- `util.content-to-tree()`
- `util.tree-to-content()`
- `util.random-color()`

`util.walk(tree, pre, post, leaf)`

Visit every node in a tree and transform it.

- **tree:** A root node in the format described in [How trees are represented](#).
- **pre:** `it ⇒ it` Function to transform each node *before* visiting its children.
- **post:** `it ⇒ it` Function to transform each node *after* its children have been visited.
- **leaf:** `it ⇒ it` Function to apply to leaf nodes, or all non-node children of nodes in the tree.

For most tasks, a post-walk is what you want (pre-walks can lead to infinite recursion if used incorrectly).

`util.element-fields-to-arguments(fn, fields)`

Convert an element's fields into an `arguments` object that can be passed to the element function to reinstantiate it as content.

This works for *almost* all elements:

```
let it = /* some content */
let (fn, fields) = (it.func(), it.fields())
assert.eq(it, fn(..element-fields-to-arguments(fn, fields)))
```

Some content functions require some fields to be given as positional or variadic arguments. This function takes care of all this irregularity, providing an easier way to produce elements from field dictionaries.

However, there are still some elements (like `sequence` and `math.sqrt`) which need special treatment, as done by `util.content-to-tree(it, exclude)`.

`util.stringify(it)` ↑

Convert anything into a string.

For textual elements, returns the `"text"` field. Falls back to `repr` for other content.

`util.content-to-tree(it, exclude)` ↑

Convert content into an abstract syntax tree.

- **exclude:** () Names of element functions to exclude from converting as strings.

Each node in the resulting tree is of the form:

```
(
  head: "element-name",
  fn: <element-fn>,
  args: <positional-args>,
  slots: <named-args>,
)
```

The node may be converted back into content by calling `fn(..args, ..slots)`, provided its arguments and slots have also been converted from nodes into content. This is what `util.tree-to-content(tree)` does.

`util.tree-to-content(tree)` ↑

Reconstruct content from a tree given by `util.content-to-tree(it, exclude)`.

`util.random-color(seed, lightness, chroma, hue)` ↑

A pseudo-random colour based on a seed value.

Subsequent arguments are passed to `color.oklch` and may each be a single value or a pair (min, max) indicating a random value in a range. For example, `hue: (0deg, 90deg)` produces reddish hues between `0deg` and `90deg` uniformly randomly.

```
import parsely.util: random-color
"Hello world! I am random shades of blue.".split().map(word => {
  let c = random-color(word, lightness: (20%, 70%), hue: (200deg, 300deg))
  text(c, word)
}).join(" ")
```

Hello world! I am random shades of blue.